Operation and Maintenance Manual

for VLE database

Version 1.0

by Liron Chen, id 51881969

University of Aberdeen, December 2020

Revision history

This is Operation and Maintenance Manual 1.0. Changes from previous versions are listed in Table 1 below.

Version	Date	Author/Owner	Description of Change
1.0	17/12/2020	L. Chen	Baseline Version
			•
			•
			•

Table 1 versions of the Operations and Maintenance Manual

Table of Contents

1. Introduction

This database is a basic Virtual Learning Environment. Students can enrol in courses and engage with material that education staff has uploaded. Assignments can be set with posted due dates, finished work can be handed in and marked by instructors, the resulting marks can be accessed and when marks for all assignments have been set, a grade for the course can be derived from them.

2. System Overview

The database has been progressively developed through an Entity-Relationship, Logical, and Physical model

2.1 The ER model

The conceptual or entity-relationship model can be seen in Appendix 1. For full details, the user is referred to the design report 'Designing a database for a VLE' by the author. The entities and relationships will be discussed in the discussion on the logical and physical models.

2.1.1 Core concepts

The ER model reflects the required functionality of the VLE. The underlying criteria that have determined its scope are:

- The entity staff has two subclasses: coordinators and teacher.
- They are optional and non-exclusive.
- Courses are associated with any number of instructors and coordinators or none.
- Course materials and teaching sessions have one author and associate with one course.
- Students enrol in any number of courses.
- Courses have any number of students enrolled.
- Students author any number of Turnitins (coursework to be marked).
- Turnitins have a single author, are tied to a single assignment and are marked by a single teacher.
- Marks have a weighting by which the final grade for a course is calculated
- Aggregation and composition aggregation relationships exist for coursework and marks vs calculated grades, respectively.
- Grades are derived from marks, and without them a grade cannot exist. Their relationship is a composition aggregation.
- The relationship between grades and courses has been created for more efficient querying.

2.2 The logical model

The logical model has been written as text, since the conceptual phase provides the visual overview of entities and relationships, and development of the database at this stage is facilitated by easy assembly of quick modification of elements. It can be

1:*		1:1		*:*			Complex
Students	*Turnitins*	*Teachers*	*Marks*	*Students*	*Courses*		*Teachers* *Turnitins* *Marks*
Students	*Grades*	*Courses*	*Turnitins*	*Teachers*	*Courses*		
Teachers	*Assignments*	*Assignments*	*Turnitins*				
Teachers	*Lectures*	*Turnitins*	*Marks*				
Teachers	*Reading*						
Teachers	*OLTeaching*						
Coordinators	*Courses*						
Courses	*Assignments*						
Courses	*Lectures*						
Courses	*Reading*						
Courses	*OLTeaching*						
Grades	*Marks*						

reviewed in Appendix 3. A table with all relations and their attributes has been included in Appendix 4.

2.2.1 Entities

All strong entities can be seen in the table in Appendix 4. The model does not have any weak entities.

2.2.2 Relationship types

Table 2 shows the relationship types that occur in the model:

Table 2: relationship types in the database. All 1:1 relationship types exhibit mandatory participation on only one side. The left column of each type designated the parent relation. The column 'Column' holds three variables that exhibit a ternary relationship.

2.2.2.1 One-to-many

The primary keys of the parent relations of one-to-many relationship types have been included in the child relations as foreign keys.

2.2.2.2 One-to-one

The primary keys of the parent relations of one-to-one relationship types have been included in the child relations as foreign keys.

2.2.2.3 Super- and subclasses

The entities coordinator and teacher are subclasses of the superclass staff; staff is therefore the parent entity. The relationship is optional and nondisjoint, since

a member of staff can belong to neither, one, or both subclasses. The subclasses have their own distinct relationships and will be crucial in determining access privileges (see 2.3.6.4). To ensure correct identification and prevent inadvertent cross class joining the subclasses are assigned their own relations, with the staff primary key as a foreign key.

2.2.2.4 Many-to-many

For each of the two many-to-many relationships a table has been created, Enrolment and Instructors respectively, that uses the primary keys of both associated relations as a composite primary key and as separate foreign keys. The other method of converting the *:* to two 1:* types and adding the primary keys as foreign keys in the each opposite table generates duplicate data by repeating student records for each course enrolled in and course information for each enrolled student, and is deemed less suitable.

2.2.2.5 Complex, ternary

The ternary relationship has been represented in a new table Marking that uses the primary keys of all three associated relations in a composite primary key and refers to them separately as foreign keys.

2.2.3 Integrity constraints

The primary keys are mandatory. Foreign keys are mandatory in the case of the course materials, Turnitins, grades and marks: this guards course materials from being orphaned from their author or course and grades from being untraceable to their author or marking teacher. All other foreign keys and attributes are optional, to allow for multi-session data entry. Incomplete data may occur but can be seen on any query and does not pose a threat to database integrity.

Although domain constraints are useful for this database, they have not been applied due to physical limitations, refer to section 2.3.3 for details.

2.3 The physical model

The physical model, in the form of SQL coding, can be seen in Appendix 3. It includes a list of commands to populate the database with example data.

2.3.1 Syntax

Every relation and relationship has been written in a similar format. Capitals have been used for keywords, new paragraphs indicate new relations, new lines indicate new attributes.

2.3.2 Engine

The engine that was defined for each relation is MyISAM, this is because the application used to implement the database, PHPMyAdmin, uses InnoDB as a default, and the required syntax does not function in that engine.

2.3.3 Integrity constraints

The primary keys are set to NOT NULL, as well as the foreign keys in the course materials, Turnitins, grades and marks. Refer to 2.2.3 for details.

No domain constraints on the various ID functions could be used as the MyISAM engine does not support domain functions. The definitions of the primary and foreign keys, along with their NULL and UPDATE / DELETE settings constrain them to a sufficient extent.

2.3.4 Example data

The coding that uploads data into the database holds details of current courses, students and staff from the University of Aberdeen that the author has access to, or fictional information based on the same. All relations are populated, however only the course with ID C0005098 has been given a full cast of materials, students, teachers, Turnitins, marks and grades.

2.3.5 Foreign keys

The data that foreign keys refer to can be deleted or updated. Commands that specify the action taken when that happens have been included (ON DELETE and ON UPDATE).

- Most foreign keys will SET NULL on deletion, creation parentless child relations; an example is a course without a teacher or coordinator. However course materials will apply DELETE if the course they belong to is deleted.
- All student work is update or deleted when the associated student records are.
- Foreign keys in Marks, Grades and Marking will accept updating but RESTRICT deletion of the data they refer to. This information is the most sensitive and valuable in the database; accidental deletion is prevented, and the integrity of its relationships is guarded.
- Enrolment and Instructors that represent many-to-many relationships allow foreign keys to be updated and deleted.

2.3.6 Big Data

Certain big data concepts have been considered for the database:

2.3.6.1 Spatial data extensions

MySQL supports spatial data, in fact the InnoDB engine can process spatial data since version 5.0.15 of MySQL, while MyISAM could before that. However, the current database for a VLE does not require spatial data handling and it is unlikely that it will in the future – the concept of a VLE is precisely that it can be accessed and used from anywhere and consists of a virtual environment, one that is not based in a physical location.

2.3.6.2 Time series

It can be useful to track updates and changes and analyse how the data pool is modified over time. However, most relations include date stamps in the form of attributes for relevant dates – for when data is posted, updated, tasks are due, or other significant events happen. Queries can be written to return all data uploaded for any timeframe. All changes are also tracked by the DBMS journaling service (see 3.3).

2.3.6.3 Transaction management

As can be seen in the physical model, and as detailed in section 3.2, each data

insertion or modification must be accompanied by the appropriate transaction management commands. The database been defined with AUTOCOMMIT=OFF and therefore any transaction needs to be started and committed to be applied. The selected isolation level for this database is READ COMMITTED. There is a chance of simultaneous sessions, and repeated attempts at data updating. Only completed, verified and correctly formatting transactions will be applied to the database and subsequently read by other users. Non-repeatable reads can still be useful for users – for example, a student that is logged in will want to view a newly posted course lecture without having to renew the session – and therefore stricter isolation is not desirable.

2.3.6.4 Access control

Coding for user profiles for each teacher and students was assembled, however the application used does not allow the creation of users nor the granting of privileges. Staff require access to any course they are associated with, material they have posted, any Turnitins connected to assignments they posted, and the resulting marks. Staff that are designated as coordinators require access to all teachers and their materials for the course they are coordinating. An example of coding to set access rights for one member of staff, by necessity untested, is provided in Appendix 6.

Students require access to the courses they are enrolled in, all its materials, any Turnitins they author, and the resulting marks and grades.

Views could be created for each user instead to customise what data is visible. However, since it is also imperative that privileges are controlled (such as updating marks for students), this process is deemed less suitable.

3. Operating procedure

3.1 Queries

The database can be queried for any data requirements. Example queries that show the structure of the database are shown below.

- Example: show grades for each student for each course:
  SELECT Grades.val, fName, lName, Courses.name FROM Grades, Students, Courses WHERE Grades.studentID=Students.studentID AND Grades.courseID=Courses.courseID;
- Example: show marks for course C0005098 per student:
  SELECT val, fName, lName, Courses.name FROM Marks,Students,Courses,Enrolment WHERE Students.studentID = Enrolment.studentID AND Enrolment.courseID = Courses.courseID AND Courses.courseID = 'C0005098';
- Example: show courses that member of staffs are teaching:
  SELECT Staff.fName, Staff.lName, Courses.name FROM Staff, Teachers,

Courses WHERE Courses.teacherID=Teachers.teacherID AND
Teachers.staffID=Staff.staffID

Based on these and the appendixes any query can be constructed.

3.2 Transactions

Any data insertion or modification must be issued in a transaction. Autocommit has been disabled. Use the following syntax:

START TRANSACTION;
[data operations]
COMMIT;

Isolation level is set to READ COMMITTED.

3.3 Backups and journaling

The DBMS should be enabled to perform a regularly scheduled backup for immediate recovery to the last known state in case of any failure. Journaling to log any changes made to the database needs to be active as well, to reapply any changes since the last backup, in case the backup is lost, to revert back to older states before the last backup, and to track the modifications of materials and grades. However both these functions are denied to the author with the application used at the time of writing.

3.4 Service plan

Only coordinators and the administrator have rights to delete courses and their associated materials, marks, grades, and Turnitins. Those data can be 'orphaned' when teachers designations, staff or student records are removed.

At the end of each course, an inventory of connected materials should be made, and decision about what data to keep in storage and what to remove needs to be taken. This should keep the database free from clutter.

4. Recommendations for database lifecycle support

As mentioned in 2.3.3. and 2.3.6.4, certain integrity protections and access control mechanisms were by necessity not implemented. To protect the database, these should be applied before publishing. The four stages of the database lifecycle are detailed below:

4.1 Requirements analysis

Additional functionality will be repeatedly added as the database grows its user base and the online teaching environment develops.

4.2 Design

The same cycle of conceptual / ER model, logical model, and physical model should be followed.

4.3 Implementation

Ensure that the integrity of the existing database, and in particular the marks and

grades data is not affected during data transfer to newer versions. Record the changes and new version number in the versioning table at the start of this manual.

4.4 Maintenance

Similar recommendations for maintenance would hold true for future cycles as for the current version.

5. References

- Course materials provided for Database Systems and Big Data, CS5097, CS5098, University of Aberdeen
- Connolly, T.M. and Begg, C.E., 2015. Database systems: a practical approach to design, implementation, and management, Global ed. *Harlow, Pearson Education*.

Appendix 1 the ER model

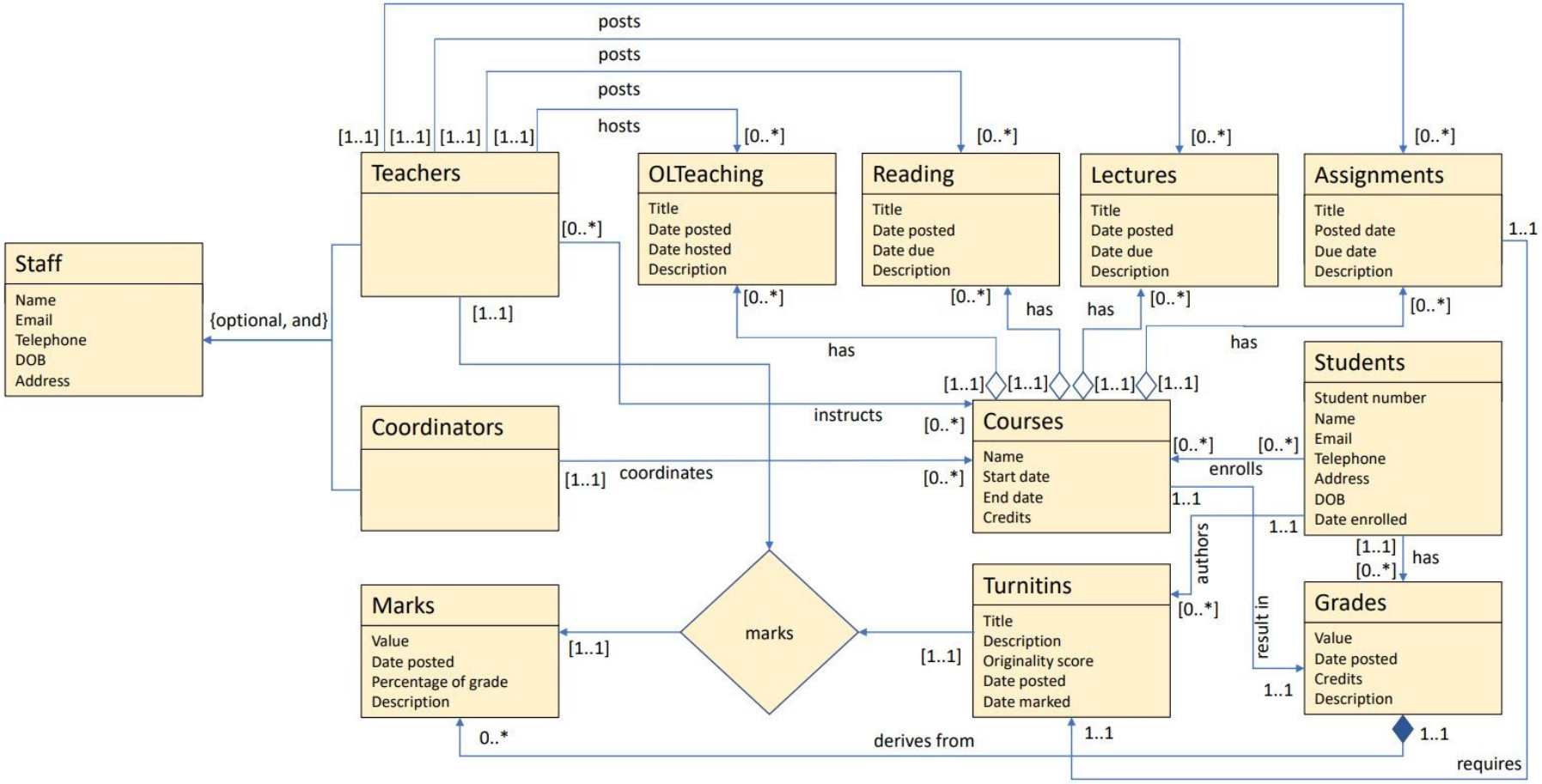

Figure 1: the ER model

Appendix 2: the logical Model

Students (studentID, Fname, Lname, email, telephone, address1, address2, city, postcode, dob)
Primary Key studentID

Staff (staffID, Fname, Lname, email, telephone, address1, address2, city, postcode, dob)
Primary Key staffID

Teachers (teacherID, staffID)
Primary Key teacherID

Coordinators (coordinatorID, staffID)
Primary Key coordinatorID

Courses (courseID, name, startDate, endDate, credits, coordinatorID)
Primary Key courseID
Foreign Key coordinatorID references Coordinators(coordinatorID)

Assignments (assignmentID, name, postedDate, dueDate, markPercentage, description, teacherID, CourseID)
Primary Key assignmentID
Foreign Key teacherID references Teachers(teacherID)
Foreign Key courseID references Courses(courseID)

Lectures (lectureID, title, postedDate, dueDate, description, teacherID, courseID)
Primary Key lectureID
Foreign Key teacherID references Teachers(teacherID)
Foreign Key courseID references Courses(CourseID)

Reading (readingID, title, postedDate, dueDate, description, teacherID, courseID)
Primary Key readingID
Foreign Key teacherID references Teachers(teacherID)
Foreign Key courseID references Courses(CourseID)

OLTeaching (olteachingID, postedDate, dateHosted, description, teacherID, courseID)
Primary Key olteachingID
Foreign Key teacherID references Teachers(teacherID)
Foreign Key courseID references Courses(courseID)

Turnitins (turnitinID, title, postedDate, originalityScore, description, studentID, assignmentID)
Primary Key turnitinID
Foreign Key studentID references Students(studentID)
Foreign Key assignmentID references Assignments(assignmentID)

Marks (markID, val, valnr, postedDate, gradePercentage, description, turnitinID)
Primary Key markID
Foreign Key turnitinID references Turnitins(turnitinID)

Grades (gradeID, val, valnr, postedDate, credits, description, studentID)
Primary Key gradeID
Foreign Key studentID references Students(studentID)

Enrolment (studentID, courseID)
Primary Key studentID, courseID
Foreign Key studentID references Students(studentID)
Foreign Key courseID references Courses(courseID)

Instructors (teacherID, courseID)
Primary Key teacherID, courseID
Foreign Key teacherID references Teachers(teacherID)
Foreign Key courseID references Courses(courseID)

Marking (turnitinID, teacherID, markID)
Primary Key turnitinID, teacherID, markID
Foreign Key turnitinID references Turnitins(turnitinID)
Foreign Key teacherID references Teachers(teacherID)
Foreign Key markID references Marks(markID)

Appendix 3 Table of relations and attributes

Relation	Primary Key	Foreign Key	Foreign Key	Attr. 1	Attr. 2	Attr. 3	Attr. 4	Attr. 5	Attr. 6	Attr. 7	Attr. 8	Attr. 9
Students	*studentID*			*fName*	*lName*	*email*	*telephone*	*address1*	*address2*	*city*	*postcode*	*dob*
Staff	*staffID*			*fName*	*lName*	*email*	*telephone*	*address1*	*address2*	*city*	*postcode*	*dob*
Teachers	*teacherID*	*staffID*										
Coordinators	*coordinatorID*	*staffID*										
Courses	*courseID*	*coordinatorID*		*name*	*startDate*	*endDate*	*credits*					
Assignments	*assignmentID*	*teacherID*	*courseID*	*name*	*postedDate*	*dueDate*	*description*					
Lectures	*lectureID*	*teacherID*	*courseID*	*title*	*postedDate*	*dueDate*	*description*					
Reading	*readingID*	*teacherID*	*courseID*	*title*	*postedDate*	*dueDate*	*description*					
OLTeaching	*olteachingID*	*teacherID*	*courseID*	*title*	*postedDate*	*dateHosted*	*description*					
Turnitins	*turnitinID*	*studentID*	*assignmentID*	*title*	*postedDate*	*origScore*	*description*					
Marks	*markID*	*turnitinID*		*val*	*valnr*	*postedDate*	*gradePerc*	*description*				
Grades	*gradeID*	*studentID*		*val*	*valnr*	*postedDate*	*credits*	*description*				
Relation	**Primary Key**	**Primary Key**	**Primary Key**									
Enrolment	*studentID*	*courseID*										
Instructors	*teacherID*	*courseID*										
Marking	*turnitinID*	*teacherID*	*markID*									

Appendix 4 the Physical Model in SQL code

```
SET AUTOCOMMIT = OFF;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;


START TRANSACTION;


CREATE TABLE Students (
studentID varchar(8) NOT NULL,
fName varchar(30) default NULL,
lName varchar(30) default NULL,
email varchar(50) default NULL,
telephone varchar(20) default NULL,
address1 varchar(50) default NULL,
address2 varchar(2050) default NULL,
city varchar(30) default NULL,
postcode varchar(10) default NULL,
dob datetime default NULL,
PRIMARY KEY (studentID)
) ENGINE=MyISAM;

CREATE TABLE Staff (
staffID varchar(8) NOT NULL,
fName varchar(30) default NULL,
lName varchar(30) default NULL,
email varchar(50) default NULL,
telephone varchar(20) default NULL,
address1 varchar(50) default NULL,
address2 varchar(2050) default NULL,
city varchar(30) default NULL,
postcode varchar(10) default NULL,
dob datetime default NULL,
PRIMARY KEY (staffID)
) ENGINE=MyISAM;

CREATE TABLE Teachers (
teacherID varchar(8) NOT NULL,
staffID varchar(8) default NULL,
PRIMARY KEY (teacherID),
FOREIGN KEY (staffID) REFERENCES Staff
ON DELETE SET NULL
```

```
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Coordinators (
coordinatorID varchar(8) NOT NULL,
staffID varchar(8) default NULL,
PRIMARY KEY (coordinatorID),
FOREIGN KEY (staffID) REFERENCES Staff
ON DELETE SET NULL
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Courses (
courseID varchar(8) NOT NULL,
name varchar(50) default NULL,
startDate datetime default NULL,
endDate datetime default NULL,
credits int(2) default NULL,
coordinatorID varchar(8) default NULL,
PRIMARY KEY (courseID),
FOREIGN KEY (coordinatorID) REFERENCES Coordinators
ON DELETE SET NULL
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Assignments (
assignmentID varchar(8) NOT NULL,
name varchar(50) default NULL,
postedDate datetime default NULL,
dueDate datetime default NULL,
description varchar(2048) default NULL,
teacherID varchar(8) NOT NULL,
courseID varchar(8) NOT NULL,
PRIMARY KEY (assignmentID),
FOREIGN KEY (teacherID) REFERENCES Teachers
ON DELETE SET NULL
ON UPDATE CASCADE,
FOREIGN KEY (courseID) REFERENCES Courses
ON DELETE CASCADE
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Lectures (
lectureID varchar(8) NOT NULL,
title varchar(50) default NULL,
```

```
postedDate datetime default NULL,
dueDate datetime default NULL,
description varchar(2048) default NULL,
teacherID varchar(8) NOT NULL,
courseID varchar(8) NOT NULL,
PRIMARY KEY (lectureID),
FOREIGN KEY (teacherID) REFERENCES Teachers
ON DELETE SET NULL
ON UPDATE CASCADE,
FOREIGN KEY (courseID) REFERENCES Courses
ON DELETE CASCADE
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Reading (
readingID varchar(8) NOT NULL,
title varchar(50) default NULL,
postedDate datetime default NULL,
dueDate datetime default NULL,
description varchar(2048) default NULL,
teacherID varchar(8) NOT NULL,
courseID varchar(8) NOT NULL,
PRIMARY KEY (readingID),
FOREIGN KEY (teacherID) REFERENCES Teachers
ON DELETE SET NULL
ON UPDATE CASCADE,
FOREIGN KEY (courseID) REFERENCES Courses
ON DELETE CASCADE
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE OLTeaching (
olteachingID varchar(8) NOT NULL,
title varchar(50) default NULL,
postedDate datetime default NULL,
dueDate datetime default NULL,
description varchar(2048) default NULL,
teacherID varchar(8) NOT NULL,
courseID varchar(8) NOT NULL,
PRIMARY KEY (olteachingID),
FOREIGN KEY (teacherID) REFERENCES Teachers
ON DELETE SET NULL
ON UPDATE CASCADE,
FOREIGN KEY (courseID) REFERENCES Courses
```

```
ON DELETE CASCADE
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Turnitins (
turnitinID varchar(8) NOT NULL,
title varchar(50) default NULL,
postedDate datetime default NULL,
origScore int(2) default NULL,
description varchar(2048) default NULL,
studentID varchar(8) NOT NULL,
assignmentID varchar(8) NOT NULL,
PRIMARY KEY (turnitinID),
FOREIGN KEY (studentID) REFERENCES Students
ON DELETE CASCADE
ON UPDATE CASCADE,
FOREIGN KEY (assignmentID) REFERENCES Assignments
ON DELETE CASCADE
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Grades (
gradeID varchar(8) NOT NULL,
val varchar(2) default NULL,
valnr int(2) default 0,
postedDate datetime default NULL,
credits int(2) default NULL,
description varchar(2048) default NULL,
studentID varchar(8) NOT NULL,
courseID varchar(8) NOT NULL,
PRIMARY KEY (gradeID),
FOREIGN KEY (studentID) REFERENCES Students
ON DELETE RESTRICT
ON UPDATE CASCADE,
FOREIGN KEY (courseID) REFERENCES Courses
ON DELETE RESTRICT
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Marks (
markID varchar(8) NOT NULL,
val varchar(2) default NULL,
valnr int(2) default 0,
postedDate datetime default NULL,
```

```
gradePerc int(2) default NULL,
description varchar(2048) default NULL,
turnitinID varchar(8) NOT NULL,
gradeID varchar(8) NOT NULL,
PRIMARY KEY (markID),
FOREIGN KEY (turnitinID) REFERENCES Turnitins
ON DELETE RESTRICT
ON UPDATE CASCADE,
FOREIGN KEY (gradeID) REFERENCES Grades
ON DELETE RESTRICT
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Enrolment (
studentID varchar(8) NOT NULL,
courseID varchar(8) NOT NULL,
PRIMARY KEY (studentID, courseID),
FOREIGN KEY (studentID) REFERENCES Students
ON DELETE CASCADE
ON UPDATE CASCADE,
FOREIGN KEY (courseID) REFERENCES Courses
ON DELETE CASCADE
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Instructors(
teacherID varchar(8) NOT NULL,
courseID varchar(8) NOT NULL,
PRIMARY KEY (teacherID, courseID),
FOREIGN KEY (teacherID) REFERENCES Teachers
ON DELETE CASCADE
ON UPDATE CASCADE,
FOREIGN KEY (courseID) REFERENCES Courses
ON DELETE CASCADE
ON UPDATE CASCADE
) ENGINE=MyISAM;

CREATE TABLE Marking (
turnitinID varchar(8) NOT NULL,
teacherID varchar(8) NOT NULL,
markID varchar(8) NOT NULL,
PRIMARY KEY (turnitinID, teacherID, markID),
FOREIGN KEY (turnitinID) REFERENCES Turnitins
ON DELETE RESTRICT
```

```
ON UPDATE CASCADE,
FOREIGN KEY (teacherID) REFERENCES Teachers
ON DELETE RESTRICT
ON UPDATE CASCADE,
FOREIGN KEY (markID) REFERENCES Marks
ON DELETE RESTRICT
ON UPDATE CASCADE
) ENGINE=MyISAM;

COMMIT;
```

Appendix 5 Example data upload

```
START TRANSACTION;


INSERT INTO Students VALUES (00000001,'Siu', 'REDACTED', 'REDACTED@abdn.ac.uk',
        '+447900000001', 'Student Drive','Room 1', 'Aberdeen','AB24 3SW', '1991-01-01
12:00:00');
INSERT INTO Students VALUES (00000002,'Deji','REDACTED','REDACTED@abdn.ac.uk',
'+447900000002', 'Student Drive','Room 2', 'Aberdeen','AB24 3SW', '1992-01-01
12:00:00');
INSERT INTO Students VALUES (00000003,'Haihong', 'REDACTED',
'REDACTED@abdn.ac.uk',     '+447900000003', 'Student Drive','Room 3',
'Aberdeen','AB24 3SW', '1993-01-01 12:00:00');
INSERT INTO Students VALUES (51881969,'Liron', 'REDACTED', 'REDACTED@abdn.ac.uk',
        '+447900000004', 'Student Drive','Room 4', 'Aberdeen','AB24 3SW', '1994-01-01
12:00:00');

INSERT INTO Staff VALUES ('Beac0001','Nigel', 'REDACTED', 'REDACTED@abdn.ac.uk','
+44 (0)1224 273878', 'REDACTED','Department of Computing Science Meston Building',
'Aberdeen','AB24 3UE','1980-01-01');
INSERT INTO Staff VALUES ('Gree0001','David', 'REDACTED', 'REDACTED@abdn.ac.uk','
+44 (0)1224 272324', 'REDACTED','St. Mary's, Elphinstone Road', 'Aberdeen','AB24
3UF','1980-01-01');
INSERT INTO Staff VALUES ('REDACTED','Daniela', 'REDACTED',
'REDACTED@abdn.ac.uk',' +44 (0)1224 273856', 'School of Geosciences','University of
Aberdeen', 'Aberdeen','AB24 3UE','1980-01-01');
INSERT INTO Staff VALUES ('REDACTED','Matteo', 'REDACTED',
'REDACTED@abdn.ac.uk','+44 (0)1224 273034', 'Dept of Geography & Environment
School of Geoscience','Elphinstone Road', 'Aberdeen','AB24 3UF','1980-01-01');

INSERT INTO Teachers VALUES ('BeacT001','Beac0001');
INSERT INTO Teachers VALUES ('GreeT001','Gree0001');
INSERT INTO Teachers VALUES ('SpagT001','Spag0001');

INSERT INTO Coordinators VALUES ('BeacC001', 'Beac0001');
INSERT INTO Coordinators VALUES ('GreeC001', 'Gree0001');
INSERT INTO Coordinators VALUES ('GreeC002', 'Gree0001');
INSERT INTO Coordinators VALUES ('GreeC003', 'Gree0001');
INSERT INTO Coordinators VALUES ('GreeC004', 'Gree0001');

INSERT INTO Courses VALUES ('C0005098','Database Systems and Big Data','2020-09-
01','2020-12-21',15,'BeacC001');
INSERT INTO Courses VALUES ('C0005012','Dissertation Project in GIS (Online)','2021-
05-01','2021-7-21',60,'GreeC001');
```

INSERT INTO Courses VALUES ('C0005070','WebGIS and Online Mapping (Online)','2020-09-01','2020-12-21',15,'GreeC001');
INSERT INTO Courses VALUES ('C0005563','Current Applications of GIS (Online)','2020-01-01','2020-4-21',15,'GreeC001');
INSERT INTO Courses VALUES ('C0005570','Fundamentals and Advanced Applications of Map Algebra','2018-09-01','2018-12-21',15,'GreeC001');

INSERT INTO Assignments VALUES ('A0000001', 'CA1','2020-10-19','2020-10-23', 'MCQs', 'BeacT001', 'C0005098');
INSERT INTO Assignments VALUES ('A0000002', 'CA2','2020-10-19','2020-11-13', 'SQL', 'BeacT001', 'C0005098');
INSERT INTO Assignments VALUES ('A0000003', 'CA3','2020-11-16','2020-11-27', 'Database Design', 'BeacT001', 'C0005098');
INSERT INTO Assignments VALUES ('A0000004', 'CA4','2020-30-11','2020-12-18', 'Database Implementation', 'BeacT001', 'C0005098');

INSERT INTO Lectures VALUES ('L0000001', 'Lecture 0','2020-09-01','2020-12-21','Course Overview', 'BeacT001', 'C0005098');
INSERT INTO Lectures VALUES ('L0000002', 'Lecture 1','2020-09-01','2020-12-21','Introduction', 'BeacT001', 'C0005098');
INSERT INTO Lectures VALUES ('L0000003', 'Lecture 2','2020-09-01','2020-12-21','Relational Model', 'BeacT001', 'C0005098');
INSERT INTO Lectures VALUES ('L0000004', 'Lecture 3','2020-09-01','2020-12-21','SQL I', 'BeacT001', 'C0005098');
INSERT INTO Lectures VALUES ('L0000005', 'Lecture 4','2020-09-01','2020-12-21','SQL II', 'BeacT001', 'C0005098');
INSERT INTO Lectures VALUES ('L0000006', 'Lecture 5','2020-09-01','2020-12-21','Writing SQL', 'BeacT001', 'C0005098');
INSERT INTO Lectures VALUES ('L0000007', 'Lecture 6','2020-09-01','2020-12-21','Database Design', 'BeacT001', 'C0005098');
INSERT INTO Lectures VALUES ('L0000008', 'Extra Lecture ','2020-09-01','2020-12-21','SQL vs noSQL', 'BeacT001', 'C0005098');

INSERT INTO Reading VALUES ('R0000001', 'Chapter 1','2020-10-19','2020-10-30','From: Database systems: A practical approach to design, implementation, and management by T. Connolly & C. Begg, 6th Ed., Addison-Wesley (2015). Available online via the University\'s library', 'BeacT001', 'C0005098');
INSERT INTO Reading VALUES ('R0000002', 'Chapters 2-4','2020-10-19','2020-10-30','From: Database systems: A practical approach to design, implementation, and management by T. Connolly & C. Begg, 6th Ed., Addison-Wesley (2015). Available online via the University\'s library', 'BeacT001', 'C0005098');
INSERT INTO Reading VALUES ('R0000003', 'Chapters 6-9','2020-10-19','2020-11-20','From: Database systems: A practical approach to design, implementation, and management by T. Connolly & C. Begg, 6th Ed., Addison-Wesley (2015). Available online

```
via the University\'s library', 'BeacT001', 'C0005098');
INSERT INTO Reading VALUES ('R0000004', 'Chapters 10-12','2020-10-19','2020-11-
20','From: Database systems: A practical approach to design, implementation, and
management by T. Connolly & C. Begg, 6th Ed., Addison-Wesley (2015). Available online
via the University\'s library', 'BeacT001', 'C0005098');
INSERT INTO Reading VALUES ('R0000005', 'Chapters 11,12,16','2020-10-19','2020-11-
20','From: Database systems: A practical approach to design, implementation, and
management by T. Connolly & C. Begg, 6th Ed., Addison-Wesley (2015). Available online
via the University\'s library', 'BeacT001', 'C0005098');
INSERT INTO Reading VALUES ('R0000006', 'Chapters 14,17,20,21','2020-10-19','2020-
12-11','From: Database systems: A practical approach to design, implementation, and
management by T. Connolly & C. Begg, 6th Ed., Addison-Wesley (2015). Available online
via the University\'s library', 'BeacT001', 'C0005098');

INSERT INTO OLTeaching VALUES ('OLT00001','Worksession 1','2020-10-19','2020-10-
29','Questions and answers about the course, planning and work distribution',
        'BeacT001', 'C0005098');
INSERT INTO OLTeaching VALUES ('OLT00002','Worksession 2','2020-10-02','2020-11-
12','Questions and answers about SQL and database design', 'BeacT001', 'C0005098');
INSERT INTO OLTeaching VALUES ('OLT00003','Worksession 3','2020-10-16','2020-11-
26','Questions and answers about database implementation, security and views',
'BeacT001', 'C0005098');

INSERT INTO Turnitins VALUES ('TUR00001','CA1','2020-10-21', '98','Here are the
answers for my CA1 assignment.',00000001,'A0000001');
INSERT INTO Turnitins VALUES ('TUR00002','CA1 handin','2020-10-24', '98','The handin
for CA1!',00000002,'A0000001');
INSERT INTO Turnitins VALUES ('TUR00003','Work for CA1','2020-10-27',
'93','-',00000003,'A0000001');
INSERT INTO Turnitins VALUES ('TUR00004','CA1 answers','2020-10-22', '98','See
attached file, my answers for CA1.',51881969,'A0000001');
INSERT INTO Turnitins VALUES ('TUR00005','CA2','2020-11-11', '93','The answers for my
CA2.',00000001,'A0000002');
INSERT INTO Turnitins VALUES ('TUR00006','Handin for CA2','2020-11-12', '93','Handin
for CA2',00000002,'A0000002');
INSERT INTO Turnitins VALUES ('TUR00007','CA2 work ','2020-11-15',
'93','SQL',00000003,'A0000002');
INSERT INTO Turnitins VALUES ('TUR00008','CA2','2020-11-13', '93','Attached, my files
for CA2.',51881969,'A0000002');
INSERT INTO Turnitins VALUES ('TUR00009','CA3','2020-11-25', '96','Answers for
CA3.',00000001,'A0000003');
INSERT INTO Turnitins VALUES ('TUR00010','CA3 handin','2020-11-27', '95','Handin
CA3',00000002,'A0000003');
INSERT INTO Turnitins VALUES ('TUR00011','Work for CA3','2020-11-26', '94','DB
```

```
design',00000003,'A0000003');
INSERT INTO Turnitins VALUES ('TUR00012','CA3 handin','2020-12-03', '93','Files for
CA3.',51881969,'A0000003');
INSERT INTO Turnitins VALUES ('TUR00013','CA4','2020-12-17',
'91','CA4',00000001,'A0000004');
INSERT INTO Turnitins VALUES ('TUR00014','Handin for CA4','2020-12-22', '92','The
handin for CA1!',00000002,'A0000004');
INSERT INTO Turnitins VALUES ('TUR00015','CA4 work','2020-12-16', '93','DB
impl.',00000003,'A0000004');
INSERT INTO Turnitins VALUES ('TUR00016','Database Implementation','2020-12-18',
'94','CA4 files.',51881969,'A0000004');

INSERT INTO Grades VALUES ('G0000001','B3',15,'2020-01-08', '15',
'Meritorious',00000001,'C0005098');
INSERT INTO Grades VALUES ('G0000002','A4',19,'2020-01-08', '15',
'Exemplary',00000002,'C0005098');
INSERT INTO Grades VALUES ('G0000003','B3',15,'2020-01-08', '15',
'Commendable',00000003,'C0005098');
INSERT INTO Grades VALUES ('G0000004','B1',17,'2020-01-08', '15',
'Praiseworthy',51881969,'C0005098');

INSERT INTO Marks VALUES ('M0000001','A3',20,'2020-11-10', '25', 'Well
done','TUR00001','G0000001');
INSERT INTO Marks VALUES ('M0000002','A4',19,'2020-11-10', '25', 'Good
stuff','TUR00002','G0000001');
INSERT INTO Marks VALUES ('M0000003','B1',17,'2020-11-10', '25', 'Solid
effort','TUR00003','G0000001');
INSERT INTO Marks VALUES ('M0000004','A2',21,'2020-11-10', '25',
'Good','TUR00004','G0000001');
INSERT INTO Marks VALUES ('M0000005','C2',13,'2020-11-24', '25', 'Reasoning could be
clearer','TUR000055','G0000002');
INSERT INTO Marks VALUES ('M0000006','C1',14,'2020-11-24', '25', 'Some good
points','TUR00006','G0000002');
INSERT INTO Marks VALUES ('M0000007','B2',16,'2020-11-24', '25', 'Keep it
up!','TUR00007','G0000002');
INSERT INTO Marks VALUES ('M0000008','B3',15,'2020-11-24', '25', 'Good
structure','TUR00008','G0000002');
INSERT INTO Marks VALUES ('M0000009','C3',12,'2020-12-08', '25', 'Some room for
improvement','TUR00009','G0000003');
INSERT INTO Marks VALUES ('M0000010','A2',21,'2020-12-08', '25',
'Bravo','TUR00010','G0000003');
INSERT INTO Marks VALUES ('M0000011','D1',11,'2020-12-08', '25', 'You missed a
part..','TUR00011','G0000003');
INSERT INTO Marks VALUES ('M0000012','B3',15,'2020-12-08', '25', 'Generally well
```

```
done','TUR00012','G0000003');
INSERT INTO Marks VALUES ('M0000013','B2',16,'2021-01-05', '25', 'Well designed,
report could be better','TUR00013','G0000004');
INSERT INTO Marks VALUES ('M0000014','A2',21,'2021-01-05', '25',
'Outstanding!','TUR00014','G0000004');
INSERT INTO Marks VALUES ('M0000015','B1',17,'2021-01-05', '25', 'Its all
here','TUR00015','G0000004');
INSERT INTO Marks VALUES ('M0000016','A5',18,'2021-01-05', '25', 'Just missing the Big
Data elements..','TUR00016','G0000004');

INSERT INTO Enrolment VALUES (00000001,'C0005098');
INSERT INTO Enrolment VALUES (00000001,'C0005070');
INSERT INTO Enrolment VALUES (00000002,'C0005563');
INSERT INTO Enrolment VALUES (00000002,'C0005570');
INSERT INTO Enrolment VALUES (00000003,'C0005012');
INSERT INTO Enrolment VALUES (00000003,'C0005090');
INSERT INTO Enrolment VALUES (51881969,'C0005098');
INSERT INTO Enrolment VALUES (51881969,'C0005012');

INSERT INTO Instructors VALUES ('BeacT001','C0005098');
INSERT INTO Instructors VALUES ('GreeT001','C0005012');
INSERT INTO Instructors VALUES ('GreeT001','C0005070');
INSERT INTO Instructors VALUES ('GreeT001','C0005563');
INSERT INTO Instructors VALUES ('SpagT001','C0005570');
INSERT INTO Instructors VALUES ('GreeT001','C0005570');

INSERT INTO Marking VALUES ('TUR00001','BeacT001','M0000001');
INSERT INTO Marking VALUES ('TUR00002','BeacT001','M0000002');
INSERT INTO Marking VALUES ('TUR00003','BeacT001','M0000003');
INSERT INTO Marking VALUES ('TUR00004','BeacT001','M0000004');
INSERT INTO Marking VALUES ('TUR00005','BeacT001','M0000005');
INSERT INTO Marking VALUES ('TUR00006','BeacT001','M0000006');
INSERT INTO Marking VALUES ('TUR00007','BeacT001','M0000007');
INSERT INTO Marking VALUES ('TUR00008','BeacT001','M0000008');
INSERT INTO Marking VALUES ('TUR00001','BeacT001','M0000009');
INSERT INTO Marking VALUES ('TUR00010','BeacT001','M0000010');
INSERT INTO Marking VALUES ('TUR00011','BeacT001','M0000011');
INSERT INTO Marking VALUES ('TUR00012','BeacT001','M0000012');
INSERT INTO Marking VALUES ('TUR00013','BeacT001','M0000013');
INSERT INTO Marking VALUES ('TUR00014','BeacT001','M0000014');
INSERT INTO Marking VALUES ('TUR00015','BeacT001','M0000015');
INSERT INTO Marking VALUES ('TUR00016','BeacT001','M0000016');

COMMIT;
```

Appendix 5 Access privileges

START TRANSACTION;

CREATE USER 'N_Beacham'@'localhost' IDENTIFIED BY 'password';

GRANT SELECT, INSERT, UPDATE ON Assignments, Lectures, Reading, OLTeaching, Marking, Marks
WHERE Staff.lName = 'Beacham' AND
Staff.staffID = Teachers.staffID AND
Teachers.teacherID = Assignments.teacherID AND
Teachers.teacherID = Lectures.teacherID AND
Teachers.teacherID = Reading.teacherID AND
Teachers.teacherID = OLTeaching.teacherID AND
Teachers.teacherID = Marking.teacherID AND
Marking.markID = Marks.markID ;

GRANT SELECT ON Staff, Teachers, Turnitins, Instructors, Coordinators;

GRANT SELECT, INSERT, UPDATE, DELETE ON Courses, Assignments, Lectures, Reading, OLTeaching, Marking, Marks
WHERE Staff.lName = 'Beacham' AND
Staff.staffID = Coordinators.staffID AND
Coordinators.coordinatorID = Instructors.coordinatorID AND
Instructors.courseID = Courses.courseID AND
Courses.courseID = Assignments.courseID AND
Courses.courseID = Lectures.courseID AND
Courses.courseID = Reading.courseID AND
Courses.courseID = OLTeaching.courseID AND
Courses.courseID = Grades.courseID;
Instructors.teacherID = Teachers.teacherID AND
Courses.courseID = Marking.teacherID AND
Marking.markID = Marks.markID AND
Courses.courseID = Grades.courseID;

COMMIT;